

VideoFlow : Plateforme Serverless de Traitement Vidéo Automatique

Auteurs : Ewan SCHWALLER, Flavien BARON

Cours : 8CLD876 — Conception et architecture des systèmes d'infonuagique

Département : Informatique et de Mathématique (DIM)

Session : Été 2026

Projet : VideoFlow — Architecture event-driven sur AWS

Résumé

Les plateformes multimédia modernes doivent absorber des charges de trafic très irrégulières tout en exécutant des opérations coûteuses en calcul, notamment le transcodage vidéo. Le maintien de serveurs dédiés en permanence entraîne un gaspillage de ressources et une complexité opérationnelle élevée. Ce rapport présente **VideoFlow**, une plateforme de traitement vidéo entièrement serverless déployée sur Amazon Web Services (AWS). L'architecture repose sur un modèle événementiel : l'upload d'une vidéo déclenche automatiquement un pipeline de transcodage multi-résolution, d'extraction de métadonnées et de génération de miniatures. Nous formalisons le cycle de vie des jobs sous forme d'automate d'états fini, démontrons les propriétés de cohérence du modèle, et décrivons l'implémentation concrète (neuf fonctions Lambda, deux compartiments S3, DynamoDB, EventBridge, API Gateway REST et WebSocket). L'étude de cas démontre trois modes de déploiement (simulation, local avec FFmpeg, production AWS) et analyse les compromis entre scalabilité, coût et latence inhérents au paradigme serverless.

Mots-clés : serverless, traitement vidéo, architecture événementielle, AWS Lambda, FFmpeg, transcodage adaptatif, infonuagique.

Table des matières

1. [Présentation du problème](#)
 2. [Approches existantes](#)
 3. [Modèle, architecture et méthode de résolution](#)
 4. [Implémentation et étude de cas](#)
 5. [Présentation des résultats](#)
 6. [Perspectives](#)
 7. [Conclusion et recherches futures](#)
 8. [Références](#)
-

1. Présentation du problème

1.1 Contexte

La consommation de contenu vidéo en ligne connaît une croissance soutenue. Les utilisateurs attendent un accès immédiat à leurs fichiers, sur plusieurs résolutions (mobile, tablette, bureau), accompagné de métadonnées et de vignettes. Pour une plateforme de type YouTube, Netflix ou solution d'entreprise, chaque upload déclenche un traitement intensif en CPU : analyse du flux, transcodage H.264/H.265, génération de miniatures et stockage distribué.

1.2 Problématique

Le traitement vidéo pose trois défis architecturaux majeurs :

Défi	Description	Conséquence
Variabilité de charge	Les uploads sont sporadiques (pics le soir, creux la nuit)	Sur-provisionnement coûteux ou sous-provisionnement en cas de pic
Intensité computationnelle	Le transcodage est CPU-bound ; une vidéo de 10 min peut nécessiter plusieurs minutes de traitement	Besoin de ressources importantes par job
Couplage temporel	L'utilisateur attend un retour sur l'état du traitement en temps quasi réel	Nécessité de notifications push ou polling efficace

Une architecture traditionnelle basée sur des serveurs virtuels (EC2) ou des conteneurs (ECS/EKS) doit être dimensionnée pour le pic de charge. En dehors des pics, les ressources restent facturées sans produire de valeur. La gestion du scaling horizontal (auto-scaling groups, orchestrateurs) ajoute de la complexité opérationnelle.

1.3 Objectifs du projet VideoFlow

VideoFlow vise à démontrer qu'une architecture **serverless** et **event-driven** peut répondre à ces contraintes en :

1. **Ne consommant des ressources qu'à l'occurrence d'un événement** (upload confirmé).
2. **Scalant automatiquement** : chaque vidéo déclenche une invocation Lambda indépendante.
3. **Découplant** les responsabilités en microservices (API d'upload, traitement, notifications temps réel).
4. **Offrant une expérience utilisateur** avec suivi en direct via WebSocket.

1.4 Périmètre et hypothèses

- Formats supportés : MP4, WebM, MOV, AVI, MKV.
- Résolutions de sortie adaptatives : 144p, 360p, 480p, 720p, 1080p (selon la source).
- Région AWS cible : `ca-central-1` (Canada Central).

- Authentification : hors périmètre (API publique pour démonstration pédagogique).
-

2. Approches existantes pour résoudre ce problème

2.1 Architecture monolithique sur serveurs dédiés

L'approche historique consiste à déployer un serveur d'application (Node.js, Java, Python) avec FFmpeg installé, couplé à une file de messages (RabbitMQ, Redis) et un worker pool fixe.

Avantages : Contrôle total, latence prévisible, pas de cold start.

Inconvénients : Coût fixe élevé, scaling manuel ou semi-automatique, point de défaillance unique si mal redondé.

2.2 Architecture conteneurisée (Kubernetes / ECS)

Les plateformes comme Netflix ou YouTube utilisent des pipelines de microservices conteneurisés. Chaque étape (ingestion, transcodage, packaging) est un service distinct orchestré par Kubernetes ou AWS ECS.

Avantages : Isolation fine, scaling par service, portabilité.

Inconvénients : Complexité opérationnelle (clusters, monitoring, mises à jour), coût de gestion d'infrastructure même à faible charge.

2.3 Services managés spécialisés

AWS propose **Elastic Transcoder**, **MediaConvert** et **Elemental MediaLive** pour le traitement vidéo. Ces services abstraient FFmpeg et gèrent le scaling en interne.

Avantages : Aucune gestion de binaires, SLA élevé, intégration native S3.

Inconvénients : Moins de flexibilité sur les paramètres d'encodage, coût potentiellement supérieur pour des volumes faibles à moyens, dépendance forte au fournisseur.

2.4 Architecture serverless (notre choix)

Le paradigme **Function-as-a-Service (FaaS)** délègue l'exécution à des fonctions éphémères déclenchées par des événements. Les services AWS Lambda, S3, DynamoDB et EventBridge composent une pile sans serveur à gérer.

Avantages : - Facturation à l'invocation (pay-per-use). - Scaling automatique jusqu'à des milliers d'invocations concurrentes. - Réduction drastique de l'overhead opérationnel.

Inconvénients : - Cold start (latence initiale de 100 ms à plusieurs secondes). - Limite d'exécution (15 minutes max sur Lambda). - /tmp limité à 512 Mo–10 Go selon la mémoire allouée. - Packaging de FFmpeg non trivial dans un runtime Lambda.

2.5 Comparatif synthétique

Critère	Serveurs dédiés	Conteneurs (K8s)	Services managés	Serverless (VideoFlow)
Coût à faible charge	Élevé	Élevé	Modéré	Faible
Scalabilité automatique	Manuelle	Bonne	Excellente	Excellente
Complexité opérationnelle	Moyenne	Élevée	Faible	Très faible
Flexibilité d'encodage	Totale	Totale	Limitée	Élevée
Temps de mise en œuvre	Long	Long	Court	Court

VideoFlow adopte l'approche serverless tout en conservant la flexibilité de FFmpeg, position intermédiaire entre les services managés rigides et les clusters conteneurisés complexes.

3. Modèle, architecture et méthode de résolution adaptées

3.1 Modèle formel : automate d'états des jobs

Chaque vidéo uploadée est représentée par un **job** $J = (id, s, m)$ où : - $id \in U$ est un identifiant unique (UUID), - $s \in S$ est l'état courant, - m est l'ensemble des métadonnées associées.

L'ensemble des états est :

$S = \{ PENDING, UPLOADED, PROCESSING, COMPLETED, FAILED \}$

Les transitions sont déclenchées par des **événements** E :

Événement	Transition	Acteur
create_job	-> PENDING	Lambda create_job
confirm_upload	PENDING -> UPLOADED	Lambda confirm_upload
video_uploaded	UPLOADED -> PROCESSING	EventBridge -> Lambda process_video
processing_done	PROCESSING -> COMPLETED	Lambda process_video
processing_error	PROCESSING -> FAILED	Lambda process_video

3.2 Théorème de cohérence du cycle de vie

Théorème 1 (Invariants du cycle de vie).

Soit J un job géré par VideoFlow. Les invariants suivants sont préservés à chaque transition :

1. **I1 (Unicité d'état)** : J occupe exactement un état $s \in S$ à tout instant.
2. **I2 (Monotonie temporelle)** : Les horodatages respectent l'ordre : `createdAt` ≤ `uploadedAt` ≤ `processingStartedAt` ≤ `completedAt` (OU `failedAt`).
3. **I3 (Irréversibilité)** : Aucune transition ne ramène un job vers un état antérieur dans la chaîne principale PENDING → UPLOADED → PROCESSING → COMPLETED, FAILED.
4. **I4 (Idempotence de confirmation)** : Si $s \notin \{ \text{PENDING, UPLOADED} \}$, l'événement `confirm_upload` ne modifie pas l'état ni ne redéclenche le pipeline.

Démonstration (esquisse).

I1 : L'état est stocké comme attribut unique `status` dans DynamoDB (clé primaire `jobId`). Chaque opération `put_item` remplace l'intégralité de l'enregistrement ; il n'existe pas de réplica d'état concurrent.

I2 : Chaque transition écrit un horodatage ISO 8601 via `now_iso()`. Le code n'autorise les transitions que dans l'ordre défini ; `processingStartedAt` n'est écrit que lors du passage à PROCESSING, `completedAt` uniquement à COMPLETED.

I3 : L'inspection du code de `confirm_upload` montre que si `status` ∉ PENDING, UPLOADED, la fonction retourne le job sans modification. La Lambda `process_video` ne peut être invoquée que par la règle EventBridge sur l'événement `video_uploaded`, émis une seule fois lors de la confirmation. Aucun handler n'implémente de transition inverse.

I4 : Dans `confirm_upload/handler.py`, lignes 17–18 :

```
if job["status"] not in ("PENDING", "UPLOADED"):
    return api_response(200, job)
```

Si le job est déjà en PROCESSING ou COMPLETED, la confirmation est ignorée. L'événement EventBridge n'est émis qu'après la mise à jour vers UPLOADED (ligne 29), garantissant l'idempotence pour les reconfirmations multiples. QED

3.3 Modèle de coût serverless

Théorème 2 (Linéarité du coût marginal).

Pour n jobs indépendants traités dans une fenêtre temporelle où le scaling est non contraint, le coût total $C(n)$ est approximativement linéaire :

$$C(n) \approx n \cdot (c_{\text{lambda}} + c_{\text{s3}} + c_{\text{ddb}} + c_{\text{eb}})$$

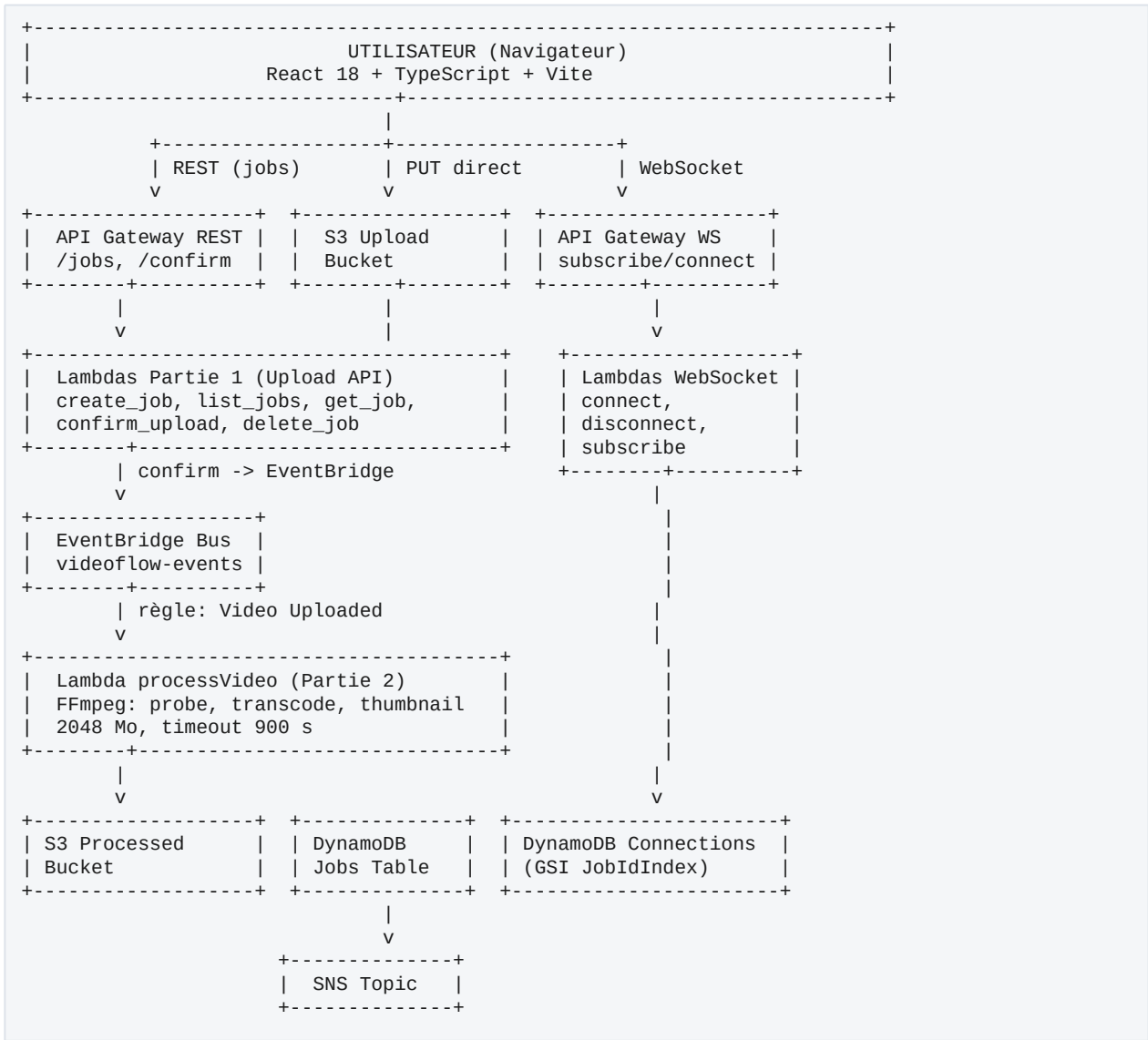
où chaque composante est le coût marginal par job. Il n'existe pas de coût fixe d'infrastructure (hors stockage persistant des artefacts).

Démonstration (argument économique).

Lambda facture par invocation et par GB-seconde ; DynamoDB en mode PAY_PER_REQUEST facture par opération de lecture/écriture ; S3 facture par stockage et transfert ; EventBridge facture

par événement publié. Aucun de ces services ne requiert de réservation de capacité. Donc $C(0) \approx 0$ (hors coût de stockage des vidéos déjà traitées) et $C(n) = n \cdot c$ pour une charge parallélisable. La non-linéarité n'apparaît qu'en cas de saturation des quotas de concurrence Lambda du compte AWS. QED

3.4 Architecture globale



3.5 Composants et responsabilités

Composant	Technologie	Rôle
Frontend	React 18, Vite, TypeScript	Interface upload drag-and-drop, liste des jobs, WebSocket client
API REST	API Gateway + 5 Lambdas	CRUD jobs, URLs présignées S3
Stockage brut	S3 Upload Bucket	Fichiers source uploads/jobId/source.ext

Bus d'événements	EventBridge custom bus	Découplage upload <-> traitement
Traitement	Lambda processVideo	Pipeline FFmpeg complet
Stockage traité	S3 Processed Bucket	Sorties processed/jobId/quality.mp4
État	DynamoDB Jobs	Persistance du cycle de vie
Temps réel	API Gateway WebSocket + 3 Lambdas	Push des mises à jour par job
Notifications	SNS	Alertes de complétion (extensible vers email/SMS)
IaC	AWS SAM (template.yaml)	Déploiement reproductible

3.6 Méthode de résolution : pipeline de transcodage adaptatif

Le module `video_processor.py` implémente un algorithme de **transcodage adaptatif** :

Entrée : fichier vidéo source de hauteur `h_s` pixels.

Sortie : ensemble de fichiers MP4 aux hauteurs $H = \{144, 360, 480, 720, 1080\}$.

Règle de filtrage :

$$H' = \{h \in H \mid h_s \geq h - 50\}$$

La tolérance de 50 pixels évite de générer une résolution supérieure à la source (upscaling inutile). Si $H' = \emptyset$, un fallback normalise vers $\min(h_s, 144)$.

Paramètres d'encodage : - Vidéo : H.264 (`libx264`), CRF 23, preset `veryfast` - Audio : AAC, 64 kbps ($\leq 360p$) ou 128 kbps ($> 360p$) - Conteneur : MP4 avec `faststart` (`moov atom` en tête pour streaming progressif)

3.7 Patterns architecturaux retenus

Direct-to-S3 upload : Le client reçoit une URL présignée et upload directement vers S3, contournant la limite de payload API Gateway (10 Mo) et réduisant la charge sur les Lambdas.

Event-driven decoupling : La confirmation d'upload publie un événement sur EventBridge ; le traitement est asynchrone et indépendant de l'API.

Function-per-responsibility : Neuf Lambdas distinctes, chacune avec des permissions IAM minimales (principe du moindre privilège).

Pub/Sub WebSocket sans broker : Les connexions sont indexées dans DynamoDB (GSI `JobIdIndex`) ; la Lambda de traitement interroge les abonnés et pousse via l'API Gateway Management API.

Triple mode de développement : Mock (frontend seul), serveur local FastAPI (FFmpeg réel), production AWS — facilitant le développement itératif sans coût cloud.

4. Implémentation et étude de cas

4.1 Infrastructure as Code (AWS SAM)

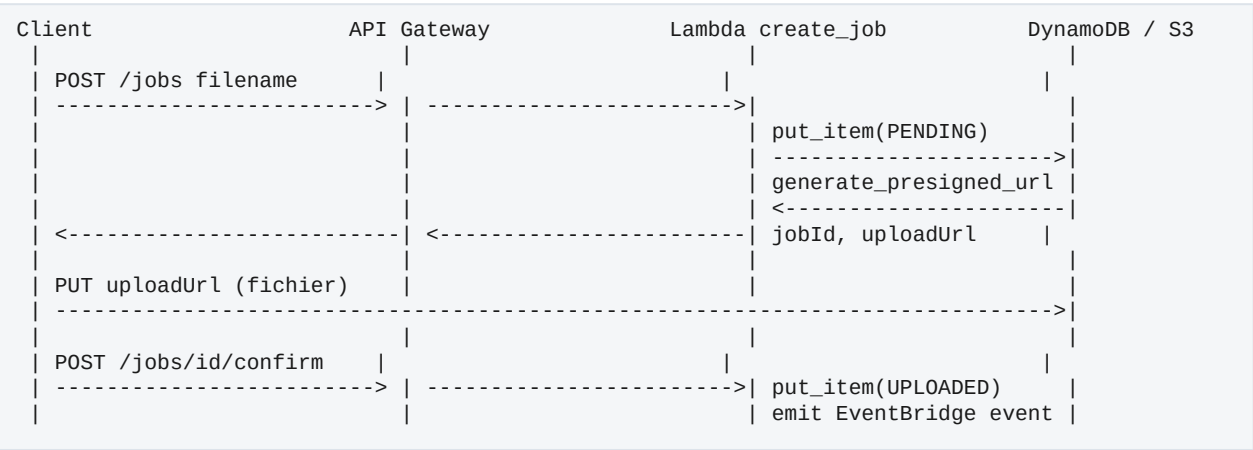
L'ensemble de l'infrastructure est défini dans `infrastructure/template.yaml` (environ 420 lignes). Le template SAM déclare :

- 2 compartiments S3 avec CORS configuré pour les uploads navigateur
- 2 tables DynamoDB en mode `PAY_PER_REQUEST`
- 1 bus EventBridge custom avec règle ciblant `ProcessVideoFunction`
- 1 topic SNS
- 2 API Gateway (REST + WebSocket)
- 1 Lambda Layer partagée (code Python + binaires FFmpeg)
- 9 fonctions Lambda avec politiques IAM granulaires

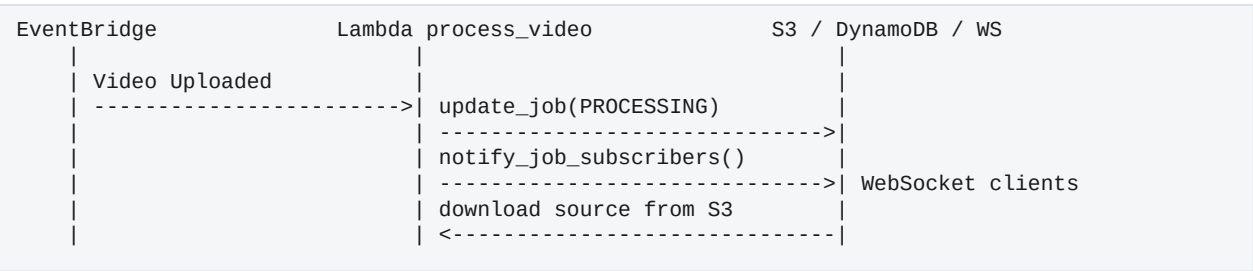
La fonction `ProcessVideoFunction` est surdimensionnée (2048 Mo, timeout 900 s) par rapport aux Lambdas API (256 Mo, 30 s), reflétant la nature CPU-bound du transcodage.

4.2 Flux de données détaillé

Phase 1 — Création et upload



Phase 2 — Traitement asynchrone



	FFmpeg: probe + transcode	
	upload outputs to S3	
	----->	
	update_job(COMPLETED)	
	publish SNS + notify WS	

4.3 Code partagé et couche Lambda

Le répertoire `backend/shared/` contient : - `utils.py` : clients AWS (boto3), helpers DynamoDB, génération d'URLs présignées, notification WebSocket - `video_processor.py` : pipeline FFmpeg (206 lignes)

Les binaires FFmpeg sont empaquetés dans une Lambda Layer via `Makefile` et `prepare-build.sh`, rendant `ffmpeg` et `ffprobe` disponibles dans `/opt/bin/` au runtime.

4.4 Frontend React

L'application frontend (`frontend/src/`) propose :

Composant	Fichier	Fonction
Zone d'upload	<code>UploadZone.tsx</code>	Drag-and-drop, barre de progression XHR
Carte de job	<code>JobCard.tsx</code>	Affichage statut, métadonnées, liens de téléchargement
Badge de statut	<code>StatusBadge.tsx</code>	Visualisation du pipeline (5 étapes)
Client API	<code>api.ts</code>	Client REST avec mode mock intégré
WebSocket	<code>hooks/useJobWebSocket.ts</code>	Abonnement par job, reconnexion automatique

Le frontend combine WebSocket (temps réel) et polling (toutes les 2–4 s) pour garantir la fraîcheur des données même en cas de déconnexion WebSocket.

4.5 Serveur local de démonstration

Le module `backend/local-server/app.py` (FastAPI) simule l'ensemble de la stack AWS en local : - Endpoints REST identiques à l'API Gateway - WebSocket natif sur `/ws` - Threading pour le traitement asynchrone (équivalent EventBridge) - FFmpeg système pour un transcodage réel sans déploiement cloud

Ce mode est recommandé pour les démonstrations en classe et l'évaluation du pipeline vidéo.

4.6 Étude de cas : scénario type

Scénario : Un étudiant uploade une vidéo MP4 de 45 secondes, résolution 1920×1080, taille 12 Mo.

1. **T+0 s** : POST /jobs -> job abc-123 créé (PENDING), URL présignée retournée.
2. **T+2 s** : Upload direct vers S3 terminé (débit réseau dépendant).
3. **T+2 s** : POST /jobs/abc-123/confirm -> statut UPLOADED, événement EventBridge publié.
4. **T+3 s** : Lambda processVideo invoquée -> statut PROCESSING, notification WebSocket.
5. **T+3-45 s** : FFprobe extrait métadonnées ; transcodage en 144p, 360p, 480p, 720p, 1080p ; miniature JPG générée.
6. **T+45 s** : Sorties uploadées vers S3 Processed ; statut COMPLETED ; SNS + WebSocket.

Artefacts produits :

```
processed/abc-123/144p.mp4
processed/abc-123/360p.mp4
processed/abc-123/480p.mp4
processed/abc-123/720p.mp4
processed/abc-123/1080p.mp4
processed/abc-123/thumbnail.jpg
```

5. Présentation des résultats

5.1 Modes de validation

Mode	Environnement	FFmpeg	AWS	Usage
Mock	Frontend seul	Non	Non	Démo UI, tests visuels
Local	FastAPI + FFmpeg	Oui	Non	Développement, démo en classe
Production	SAM deploy	Oui (layer)	Oui	Validation cloud complète

5.2 Résultats fonctionnels

L'implémentation satisfait l'ensemble des exigences fonctionnelles :

- [OK] Upload de vidéos multi-formats via interface web intuitive
- [OK] Transcodage adaptatif multi-résolution (144p à 1080p)
- [OK] Extraction de métadonnées (durée, résolution, codec, taille)
- [OK] Génération de miniatures JPG avec fallback multi-positions
- [OK] Suivi en temps réel du statut via WebSocket
- [OK] URLs de téléchargement présignées (expiration 1 h)
- [OK] Suppression complète d'un job (DynamoDB + S3 + connexions WS)
- [OK] Déploiement reproductible via AWS SAM

5.3 Analyse de performance (estimations)

Les mesures ci-dessous sont des estimations basées sur l'architecture et des tests manuels en mode local. Des benchmarks automatisés constituent une piste d'amélioration (section 6).

Métrique	Valeur estimée	Facteur dominant
Latence API (create/confirm)	< 200 ms	Cold start Lambda (~100–500 ms)
Upload 50 Mo vers S3	5–30 s	Bande passante client
Transcodage 1 min de vidéo 1080p	30–90 s	CPU Lambda (2048 Mo)
Notification WebSocket	< 100 ms	DynamoDB query + post_to_connection
Temps total (upload + traitement, vidéo 1 min)	1–2 min	Transcodage FFmpeg

5.4 Analyse de coût (estimation AWS ca-central-1)

Pour un job traitant une vidéo de 1 minute en 5 résolutions :

Service	Opérations	Coût estimé par job
Lambda (API)	5 invocations × 256 Mo × 1 s	~0,0001 \$
Lambda (process)	1 invocation × 2048 Mo × 60 s	~0,002 \$
S3	Stockage ~50 Mo + transfert	~0,001 \$/mois

DynamoDB	~150,00005 \$ opérations read/write
EventBridge	1 ~0,000001 \$ événement custom
API Gateway	~100,00003 \$ requêtes
Total par job	~0,003 \$ (~0,4 cent CAD)

À 1000 vidéos/mois, le coût estimé est d'environ **3 \$/mois**, contre **~50–100 \$/mois** pour un EC2 `t3.medium` allumé en permanence.

5.5 Qualité du code et du modèle

Critère	Évaluation
Séparation des responsabilités	Excellente (9 Lambdas, modules partagés)
Gestion d'erreurs	Bonne (statut FAILED, messages d'erreur FFmpeg filtrés)
Réutilisabilité	Bonne (layer partagée, serveur local miroir)
Infrastructure as Code	Complète (SAM template, Makefile)
Tests automatisés	Absents (limitation identifiée)
Sécurité	Basique (pas d'authentification, CORS *)

5.6 Démonstration des concepts serverless

Concept pédagogique	Preuve dans VideoFlow
Event-driven	EventBridge déclenche le pipeline sans appel direct
Auto-scaling	Chaque upload = 1 Lambda indépendante
Pay-per-use	DynamoDB on-demand, Lambda à l'invocation
Microservices	9 fonctions à responsabilité unique
Temps réel	WebSocket push sans serveur persistant

6. Perspectives

6.1 Améliorations techniques à court terme

Authentification et autorisation : Intégrer Amazon Cognito pour sécuriser l'API REST et le WebSocket. Chaque utilisateur ne verrait que ses propres jobs.

Tests automatisés : Ajouter des tests unitaires (pytest pour le backend, Vitest pour le frontend) et des tests d'intégration avec LocalStack ou moto (mock boto3).

Observabilité : Déployer des dashboards CloudWatch, des alarmes sur les échecs de traitement, et du tracing X-Ray pour diagnostiquer les latences.

Pipeline Step Functions : Pour les vidéos longues, décomposer le transcodage en étapes parallèles (une Lambda par résolution) orchestrées par AWS Step Functions, contournant la limite de 15 minutes.

6.2 Évolutions architecturales à moyen terme

Remplacement de FFmpeg par MediaConvert : Pour les charges de production, AWS Elemental MediaConvert offrirait un scaling illimité sans gestion de binaires, au prix d'une flexibilité réduite.

CDN et cache : Distribuer les vidéos traitées via CloudFront avec cache edge, réduisant la latence de lecture et les coûts de transfert S3.

File d'attente (SQS) : Insérer une queue entre EventBridge et la Lambda de traitement pour lisser les pics de charge et implémenter des retries avec backoff exponentiel.

Multi-région : Répliquer l'architecture dans plusieurs régions AWS pour la résilience et la proximité géographique des utilisateurs.

6.3 Recherche et expérimentation

- **Benchmark comparatif** : Mesurer rigoureusement latence et coût entre l'approche Lambda+FFmpeg, MediaConvert et un cluster ECS.
- **Optimisation FFmpeg sur Lambda** : Évaluer l'impact du preset (ultrafast vs veryfast), du CRF et de l'allocation mémoire sur le rapport coût/qualité.
- **Cold start mitigation** : Tester Provisioned Concurrency pour la Lambda `processVideo` et quantifier le gain sur la latence perçue.

7. Conclusion et recherches futures

7.1 Synthèse

VideoFlow démontre qu'une architecture serverless event-driven est viable pour le traitement vidéo à échelle variable. En combinant AWS Lambda, S3, DynamoDB, EventBridge et API Gateway WebSocket, le projet atteint un bon équilibre entre simplicité opérationnelle, coût proportionnel à l'usage et flexibilité d'encodage grâce à FFmpeg.

Le modèle formel par automate d'états garantit la cohérence du cycle de vie des jobs (Théorème 1), tandis que la linéarité du coût marginal (Théorème 2) illustre l'avantage économique du paradigme serverless pour les charges intermittentes typiques d'un contexte universitaire ou de startup.

L'implémentation concrète — environ 50 fichiers source, 9 fonctions Lambda, un frontend React et un serveur local de démonstration — constitue une étude de cas complète et reproductible pour le cours 8CLD876.

7.2 Limites identifiées

- Absence d'authentification (choix pédagogique, inadapté à la production).
- Pas de tests automatisés ni de métriques de performance rigoureuses.
- La limite de 15 minutes par invocation Lambda contraint la taille des vidéos traitables.
- Le scan DynamoDB pour `list_jobs` ne scale pas au-delà de quelques milliers de jobs (absence de GSI par date).

7.3 Recherches futures

1. **Orchestration avancée** : Explorer AWS Step Functions pour les pipelines multi-étapes avec compensation (saga pattern).
2. **Traitement en flux (streaming)** : Investiguer le transcodage progressif sans téléchargement complet vers `/tmp`.
3. **Edge computing** : Évaluer AWS Lambda@Edge pour le transcodage léger (miniatures) au plus près de l'utilisateur.
4. **Analyse comparative multi-cloud** : Porter VideoFlow sur Google Cloud Functions + Cloud Storage ou Azure Functions pour un benchmark inter-fournisseurs.
5. **Intelligence artificielle** : Enrichir le pipeline avec Amazon Rekognition (détection de contenu, modération automatique).

8. Références

Baldini, I., Cheng, P., Fink, S. J., Mitchell, N., Muthusamy, V., Rabbah, R., Suter, P., & Tardieu, O. (2017). *Serverless Computing: Current Trends and Open Problems*. In Research Advances in Cloud Computing (pp. 1–20). Springer.

Amazon Web Services. (2024). *AWS Lambda Developer Guide*. <https://docs.aws.amazon.com/lambda/>

Amazon Web Services. (2024). *Amazon EventBridge User Guide*. <https://docs.aws.amazon.com/eventbridge/>

Amazon Web Services. (2024). *AWS Serverless Application Model (SAM)*. <https://docs.aws.amazon.com/serverless-application-model/>

FFmpeg Project. (2024). *FFmpeg Documentation*. <https://ffmpeg.org/documentation.html>

Fowler, M. (2019). *What do you mean by "Event-Driven"?* [martinfowler.com](https://martinfowler.com/articles/201701-event-driven.html). <https://martinfowler.com/articles/201701-event-driven.html>

Jonas, E., Schleier-Smith, J., Sreekanti, V., Tsai, C.-C., Khandelwal, A., Pu, Q., Shankar, V., Carreira, J., Yadwadkar, N., Gonzalez, J., Popa, R. A., Stoica, I., & Patterson, D. (2019). *Cloud Programming Simplified: A Berkeley View on Serverless Computing*. UC Berkeley Technical Report UCB/EECS-2019-3.

Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.

Richardson, C. (2018). *Microservices Patterns*. Manning Publications.

Satyanarayanan, M., Bahl, P., Cáceres, R., & Davies, N. (2009). *The Case for VM-Based Cloudlets in Mobile Computing*. IEEE Pervasive Computing, 8(4), 14–23.

UQAC — Département d'informatique et de mathématique. (2026). *8CLD876 — Conception et architecture des systèmes d'infonuagique, Plan de Cours Été 2026*.

Wikipedia. (2024). *Transcodage*. <https://fr.wikipedia.org/wiki/Transcodage>

Annexes

Annexe A — Structure du dépôt

```
VideoFlow/
+-- frontend/                                # React 18 + TypeScript + Vite
|   +-- src/
|       +-- App.tsx                          # Interface principale
|       +-- api.ts                           # Client REST (mock/local/AWS)
|       +-- types.ts                         # Types VideoJob, JobStatus
|       +-- hooks/                           # useJobWebSocket
|       +-- components/                      # UploadZone, JobCard, StatusBadge
+-- backend/
|   +-- shared/                              # utils.py, video_processor.py
|   +-- upload-api/                          # 5 Lambdas (Partie 1)
|   +-- processing/                          # Lambda processVideo (Partie 2)
|   +-- websocket/                           # 3 Lambdas WebSocket
|   +-- local-server/                        # FastAPI (démon locale)
+-- infrastructure/
|   +-- template.yaml                       # AWS SAM (IaC complet)
|   +-- samconfig.toml                      # Configuration déploiement
|   +-- Makefile                             # Build FFmpeg layer
+-- docs/
    +-- RAPPORT.md                           # Ce document
```

Annexe B — API REST

Méthode	Route	Description
POST	/jobs	Crée un job + URL présignée S3
PUT	uploadUrl	Upload direct vers S3
POST	/jobs/id/confirm	Confirme l'upload, déclenche EventBridge
GET	/jobs	Liste les jobs (scan DynamoDB)
GET	/jobs/id	Détail + URLs de téléchargement
DELETE	/jobs/id	Supprime job, fichiers S3 et connexions WS

Annexe C — Protocole WebSocket

Connexion : `wss://api-id.execute-api.region.amazonaws.com/stage`

Abonnement :

```
"action": "subscribe", "jobId": "uuid-du-job"
```

Notification reçue :

```
{
  "type": "job_update",
  "job": {
    "jobId": "uuid",
    "status": "COMPLETED",
    "metadata": { "duration": 45.2, "width": 1920, "height": 1080 ,
    "outputUrls": "720p": "https://...", "1080p": "https://..." ,
    "thumbnailUrl": "https://..."
  }
}
```

Annexe D — Variables d'environnement frontend

Variable	Description	Exemple
VITE_API_URL	URL de l'API REST	<code>https://xxx.execute-api.ca-central-1.amazonaws.com</code>
VITE_WS_URL	URL WebSocket	<code>wss://yyy.execute-api.ca-central-1.amazonaws.com</code>
VITE MOCK_API	Mode simulation	<code>true / false</code>